Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes
Département Ingénierie des Systèmes Embarqués
Filière Smart Systems Engineering

TP 6 : Assembler la station EcoTri 4.0, déploiement multi-cibles et démonstrateur de bout en bout

Projet fédérateur EcoTri 4.0, sous-projet SP6 « Assembler la station » et jalon J6

Pr.Hassan BERBIA/Pr.Aissam BERRAHOU

2 novembre 2026

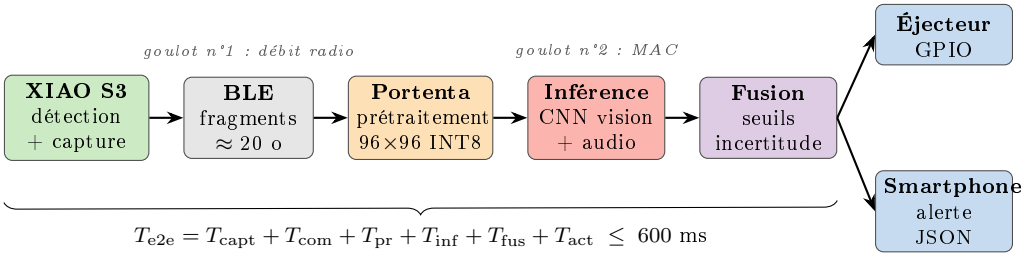
Objectifs du TP

Les cinq sous-projets précédents ont produit des briques : un protocole BLE (J3), un modèle audio (J4), un classifieur de matériaux optimisé (J5), un pipeline instrumenté (Tutorial 6). Ce TP les **assemble** en une station qui fonctionne sans PC : la XIAO détecte et capture, la Portenta décide et éjecte, le smartphone reçoit les alertes. Chaque équipe (3 ou 4 étudiants) livre un démonstrateur et le **tableau des performances du système**.
À la fin de ce TP, vous serez capable de :

- Intégrer trois nœuds hétérogènes autour d'un protocole et d'un format de message communs.
- Mesurer la latence de bout en bout et identifier le goulot par la loi d'Amdahl.
- Réduire ce goulot (imagette, recouvrement, cascade) et mesurer le gain.
- Fusionner vision et audio par règles prioritaires et gérer les objets douteux.
- Déployer le même classifieur sur la XIAO (Arduino) et comparer les deux placements du calcul.
- Valider la robustesse par un test d'endurance de 30 minutes.

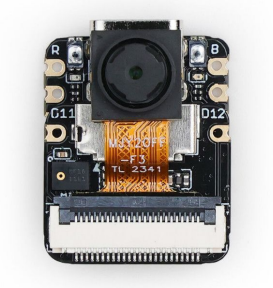
Livrable du jalon J6 (démonstration en fin de séance + dépôt) : code des trois nœuds, tableau des performances (partie 6), vidéo de 60 s, note d'architecture d'une page.

1 Contexte : la station complète

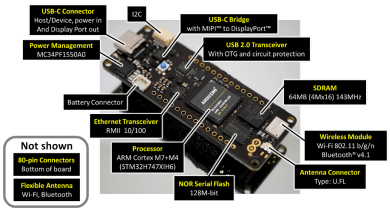


L'architecture à réaliser. Chaque flèche est une interface que l'équipe documente (format, débit, échéance).

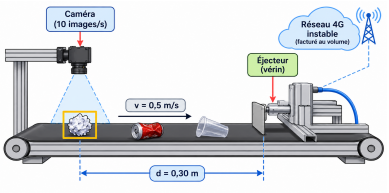
Nœud	Carte	Rôle dans ce TP
Œil du tapis	XIAO ESP32-S3 Sense	étage 1 (présence), capture 96×96 gris, envoi BLE fragmenté (protocole J3)
Unité de décision	Portenta H7 + Vision Shield	réception, classification INT8 calibrée, score audio moteur, fusion, éjecteur, alerte
Opérateur	Smartphone (nRF Connect)	réception des notifications JSON, commande « ACK »



XIAO ESP32-S3 Sense



Portenta H7



Banc de la station

Exigences du cahier des charges (J1) vérifiées dans ce TP :

Id	Exigence	Vérification
ENF-1	Latence de bout en bout ≤ 600 ms au p95, aucun dépassement non signalé	parties 2 et 3
ENF-2	Précision ≥ 95 % sur les objets triés, couverture ≥ 85 %, rappel métal ≥ 90 %	partie 4
ENF-3	Débit soutenu ≥ 1 objet/s	partie 6
ENF-4	Arrêt du tapis sur anomalie moteur en moins de 1 s	partie 4
ENF-6	30 min sans redémarrage, pertes BLE < 1 %	partie 6

2 Pré-requis

Matériel (par équipe)

- Portenta H7 + Vision Shield (avec microphones), XIAO ESP32-S3 Sense, deux câbles USB-C, un smartphone.
- Le banc du tapis, un éjecteur (servomoteur ou LED simulant l'éjecteur), 40 objets étiquetés (10 par classe).
- Un enregistrement ou une source du bruit moteur « anormal » (TP4) ; un USB-mètre si disponible.

Logiciel et fichiers

1. OpenMV IDE (firmware ≥ 4.5) ; Arduino IDE 2.x avec le paquet **esp32** d'Espressif (carte *XIAO_ESP32S3*, PSRAM « OPI PSRAM » activée).
2. Vos livrables : `ble_ecotri.py` et le sketch d'envoi de la XIAO (TP3), le modèle audio `moteur.tflite` (TP4), `trash_logits_int8.tflite`, `labels.txt`, `profil.py` et `decision.py` (Tutorial 6).
3. nRF Connect (Android ou iOS).

3 Étapes détaillées du TP

Partie 1 : contrat d'interface et chaîne minimale (25 min)

a) Figez le contrat. Avant d'écrire du code, l'équipe complète et affiche ce tableau (il fait partie du livrable) :

Interface	Format	Échéance / débit
XIAO → Portenta	en-tête (id objet, n° fragment, total, CRC-16) + charge utile ; imagette 96×96 gris	≤ . . .ms pour 9 216 o
Portenta → éjecteur	GPIO, impulsion . . .ms, délai calculé depuis `t0`	action avant . . .ms
Portenta → smartphone	JSON v1 (`id`, `etat`, `classe`, `p`, `audio`, `lat`, `mdl`)	1 notification par objet
Smartphone → Portenta	texte `ACK:<id>`, `STOP`, `START`	–

b) Côté XIAO, modifiez le sketch du TP3 pour envoyer une **imagette 96×96 en niveaux de gris** plutôt que l'image QVGA, avec l'horodatage de capture :

```
// XIAO ESP32-S3 -- capture 96x96 gris et envoi fragmenté (extrait, à intégrer au
    sketch TP3)
#include "esp_camera.h"
static uint8_t small[96 * 96];
static uint16_t objId = 0;

void captureEtEnvoi() {
  camera_fb_t* fb = esp_camera_fb_get();            // PIXFORMAT_GRAYSCALE,
    FRAMESIZE_QVGA
  const int x0 = (fb->width - fb->height) / 2;       // recadrage carré central (=
    Portenta)
  for (int y = 0; y < 96; y++)                       // sous-échantillonnage 240 ->
    96
```

```
    for (int x = 0; x < 96; x++)
      small[y * 96 + x] = fb->buf[(y * fb->height / 96) * fb->width + x0 + x * fb->
    height / 96];
  esp_camera_fb_return(fb);
  uint32_t t0 = millis();
  envoyerFragments(objId++, small, sizeof(small), t0);  // fonction du TP3 (en-tête +
     CRC)
}
```

Astuce

Le recadrage et l'échelle doivent être **identiques** à ceux de l'entraînement (Tutorial 6, étape 1). Faites le test de l'image de référence : envoyez une imagette, enregistrez-la sur la Portenta, classez-la dans Colab.

c) Côté Portenta, recevez l'imagette, reconstituez-la et classez-la :

```
# TP6-1 : réception BLE, reconstruction et classification (Portenta)
import image, ml, time
from ble_ecotri import Recepteur          # votre module du TP3 (callbacks de
    fragments)
from decision import softmax_t, decider
net = ml.Model("trash_logits_int8.tflite", load_to_fb=True)
labels = [l.strip() for l in open("labels.txt")]
img = image.Image(96, 96, image.GRAYSCALE)  # tampon pré-alloué, réutilisé
rx = Recepteur(taille=96 * 96)

while True:
    paquet = rx.image_complete(timeout_ms=400)   # (id, octets, t0) ou None
    if paquet is None: continue
    oid, data, t_recu = paquet
    img.bytearray()[:] = data                     # copie en place, sans allocation
    p = softmax_t(net.predict([img])[0].flatten().tolist())
    d, k = decider(p)
    print(oid, d, labels[k], "%.2f" % p[k])
```

Point de contrôle 1 : 10 objets consécutifs classés via la XIAO, aucun fragment perdu non détecté.

Partie 2 : profil de bout en bout et goulot (25 min)

Instrumentez la chaîne avec `Profil` sur la Portenta. Pour mesurer aussi la capture et l'envoi, la XIAO met son horodatage relatif dans l'en-tête et bascule une broche reliée à une entrée de la Portenta (front = début de capture) : la Portenta mesure ainsi `t_capture` sur sa propre horloge.

```
# TP6-2 : horodatage du début de capture par interruption (Portenta)
from pyb import ExtInt, Pin
import time
t_capture = 0
def isr(line):
    global t_capture
    t_capture = time.ticks_us()                  # pas d'allocation dans une ISR
ExtInt(Pin("D6"), ExtInt.IRQ_RISING, Pin.PULL_DOWN, isr)
# étapes : capture(t_capture) -> reçu -> prétraité -> inféré -> décidé -> éjecté
```

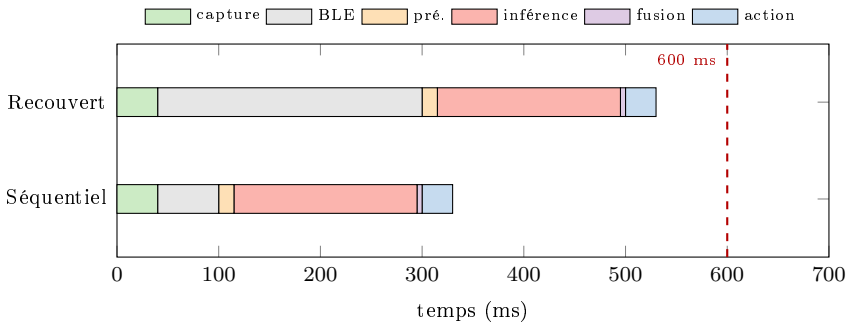
1. Mesurez 100 objets (ou 100 passages du même objet). Complétez le tableau ci-dessous et tracez le Gantt (barres empilées des médianes).
2. Calculez pour chaque étape la fraction f et le gain maximal d'Amdahl si on la divisait par 2.

Étape	Médiane (ms)	p95 (ms)	Max (ms)	Fraction f
Capture + imagette (XIAO)				
Transfert BLE				
Prétraitement / copie				
Inférence vision				
Score audio + fusion				
Action (éjecteur)				
Total				1

Partie 3 : réduire le goulot (25 min)

Appliquez **au moins deux** des optimisations suivantes, une à la fois, et remesurez après chacune :

1. **MTU et intervalle de connexion** : négocier un MTU de 247 octets et un intervalle de 7,5 ms (lecture 3) ; mesurer le nouveau débit utile.
2. **Recouvrement** : la XIAO envoie l'objet $n+1$ pendant que la Portenta classe l'objet n (double tampon côté Portenta).
3. **Cascade** : la XIAO n'envoie que si sa détection de présence (différence d'images, comme au Tutorial 6) est positive ; mesurer π_1 et le nombre d'envois évités.
4. **Éjection horodatée** : l'éjecteur agit à $t_{\text{capture}} + d/v$ (distance caméra-éjecteur sur vitesse du tapis) ; si la décision arrive trop tard, l'objet va au bac « inconnu ».

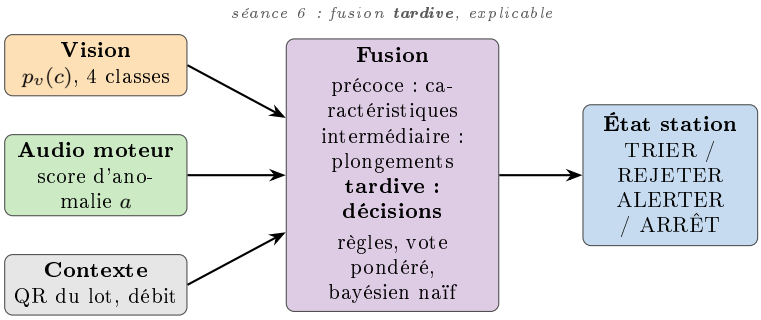


Résultat attendu : le Gantt « avant/après » de votre station, avec la droite des 600 ms.

Question

Le recouvrement améliore-t-il la latence d'un objet, le débit, ou les deux ? Justifiez avec vos mesures.

Partie 4 : fusion vision-audio et décision (25 min)



La fusion tardive par règles prioritaires : l'anomalie moteur l'emporte sur la décision de tri.

Le modèle audio du TP4 tourne sur la Portenta en tâche périodique (une fenêtre de 1 s toutes les 2 s) et met à jour un score d'anomalie lissé a. La boucle de décision lit ce score sans attendre :

```
# TP6-4 : score audio lissé, mis à jour entre deux objets
import audio, ml
mot = ml.Model("moteur.tflite")
a_lisse = 0.0
def maj_audio():
    global a_lisse
    a = score_anomalie(mot, fenetre_audio())      # fonctions de votre TP4
    a_lisse = 0.7 * a_lisse + 0.3 * a             # filtre exponentiel : évite les
    fausses alertes
    return a_lisse
# Dans DECIDER : d, k = decider(fusion_log(vues), a=a_lisse)
#   ARRET  -> LED rouge, éjecteur bloqué, notification immédiate {"etat":"ARRET", "
    audio":{...}}
```

1. Avec les 40 objets étiquetés, mesurez la précision sur les objets triés, la couverture et le rappel du métal pour $\theta \in \{0{,}6; 0{,}7; 0{,}8\}$, avec 1 puis 2 vues. Retenez le meilleur réglage au sens du coût (Tutorial 6, étape 6).
2. Jouez le son « moteur anormal » près du micro : mesurez le délai entre le début du son et la notification ARRÊT (ENF-4). Quel est l'effet du coefficient de lissage sur ce délai et sur les fausses alertes ?

Réglage	Couverture	Précision triés	Rappel métal	Erreurs	Rejets	Coût
$\theta = 0{,}6$, 1 vue						
$\theta = 0{,}7$, 1 vue						
$\theta = 0{,}7$, 2 vues						
$\theta = 0{,}8$, 2 vues						

Partie 5 : l'autre placement du calcul, inférence sur la XIAO (20 min)

Et si la XIAO classait elle-même et n'envoyait que la décision ? Déployez le même modèle sur la XIAO avec Arduino (export **Arduino library** d'Edge Impulse, ou TFLite Micro + ESP-NN avec `xxd -i`) :

```
// XIAO ESP32-S3 -- inférence locale (TFLite Micro + ESP-NN), arène en PSRAM
#include "model_int8.h"                       // xxd -i trash_logits_int8.tflite
#include <TensorFlowLite_ESP32.h>
```

```cpp
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"
constexpr size_t kArena = 300 * 1024;
uint8_t* arena;
tflite::MicroInterpreter* it;

void setupModele() {
  arena = (uint8_t*) ps_malloc(kArena);        // PSRAM : comparez avec la SRAM
    interne
  static tflite::MicroMutableOpResolver<7> ops;
  ops.AddConv2D(); ops.AddDepthwiseConv2D(); ops.AddAdd(); ops.AddMean();
  ops.AddFullyConnected(); ops.AddReshape(); ops.AddConcatenation();
  static tflite::MicroInterpreter i(tflite::GetModel(trash_logits_int8_tflite), ops,
    arena, kArena);
  it = &i;
  if (it->AllocateTensors() != kTfLiteOk) Serial.println("arène trop petite");
  Serial.printf("arène utilisée : %u o\n", it->arena_used_bytes());
}

int8_t classerLocal(const uint8_t* g, float* conf) {
  TfLiteTensor* in = it->input(0);
  const float s = in->params.scale; const int z = in->params.zero_point;
  for (int i = 0; i < 96 * 96; i++) in->data.int8[i] = (int8_t) lroundf(g[i] / 255.0f
    / s + z);
  uint32_t t = micros(); it->Invoke(); Serial.printf("inf %lu us\n", micros() - t);
  TfLiteTensor* out = it->output(0);            // logits INT8 -> softmax(z/T) comme
    sur la Portenta
  /* ... déquantifier, appliquer softmax_t, renvoyer k et conf ... */
  return 0;
}
```

1. Mesurez la latence d'inférence sur la XIAO avec l'arène en PSRAM, puis (si elle tient) en SRAM interne (`malloc` / `heap_caps_malloc(..., MALLOC_CAP_INTERNAL)`). Expliquez l'écart par le modèle roofline.
2. Complétez le tableau de comparaison des deux architectures :

Critère	**A : XIAO capture, Portenta classe**	**B : XIAO capture et classe**
Octets radio par objet		
Latence de bout en bout (p95)		
Énergie par décision côté XIAO		
Fusion avec l'audio possible ?		
Mise à jour du modèle		

Question

Quelle architecture retenez-vous pour la station, et dans quel cas changeriez-vous d'avis (autre carte, autre cadence, plusieurs caméras) ?

Partie 6 : endurance et tableau des performances, livrable J6 (25 min)

Lancez la station **30 minutes** sans PC (scripts en `main.py`, alimentation USB), avec un objet posé environ toutes les secondes et demie. Le smartphone enregistre les notifications (export du journal nRF Connect). Ajoutez dans le JSON le tas libre minimal observé.

```
# Dans NOTIFIER : télémétrie pour le tableau J6
tas_min = min(tas_min, gc.mem_free())
msg["sys"] = {"tas_min": tas_min, "perdus": rx.fragments_perdus, "uptime": time.
    ticks_ms() // 1000}
wdt.feed()                                          # machine.WDT(timeout=2000) créé au dé
    marrage
```

a) Tableau des performances du système

Indicateur	Exigence	Mesure	Méthode
Latence e2e p95 / max	≤ 600 ms		horodatages
Débit soutenu	≥ 1 objet/s		identifiants / durée
Précision triés / couverture	≥ 95 % / ≥ 85 %		40 objets étiquetés
Rappel métal	≥ 90 %		matrice de confusion
Délai d'arrêt sur anomalie	≤ 1 s		chronomètre
Pertes BLE	< 1 %		identifiants manquants
Tas libre minimal	> 50 Ko		télémétrie
Énergie par décision (XIAO)	à minimiser		USB-mètre
Redémarrages en 30 min	0		`uptime`

b) Note d'architecture (1 page) : schéma des nœuds et interfaces, machine d'états, choix d'architecture (partie 5) justifié par **trois chiffres** mesurés, une limite constatée et une piste pour la lecture 7 (mise à jour OTA, surveillance de la dérive, sécurité BLE).

c) Démonstration : 60 s de vidéo montrant un tri correct, un objet douteux envoyé au bac « inconnu » et un arrêt sur anomalie moteur.

Partie 7 (bonus) : économiser l'énergie de la XIAO

Entre deux objets, placez la XIAO en `esp_light_sleep_start()` avec réveil par minuterie (100 ms) ou par un capteur infrarouge sur GPIO. Mesurez le courant moyen avec et sans veille, calculez l'autonomie sur une batterie de 1 000 mAh et vérifiez que la latence ENF-1 reste respectée (le réveil a un coût).

4 Grille d'évaluation du jalon J6

Critère	Attendu	Points
Fonctionnement	chaîne XIAO → Portenta → éjecteur et smartphone sans PC ; arrêt sur anomalie	5
Mesures	profil de bout en bout (médiane, p95, max), tableau J6 complet, conditions déclarées	4
Optimisation	au moins deux optimisations mesurées avant/après, raisonnement d'Amdahl	3
Décision	seuil et nombre de vues choisis par le coût ; gestion du doute	3
Comparaison A/B	déploiement XIAO réalisé et comparé, choix argumenté	2
Robustesse et qualité	30 min sans redémarrage, code structuré en machine d'états, note claire	3
Total		**20**

5 Ressources

— Documentation OpenMV (`ml`, `image`, `bluetooth`, `machine.WDT`) : docs.openmv.io.

— Espressif, *esp-tflite-micro* et ESP-NN ; gestion de l'énergie ESP32-S3 : docs.espressif.com.

— Edge Impulse, déploiement Arduino et EON Compiler : docs.edgeimpulse.com.

— MLPerf Tiny (Harvard, MLCommons) et Harvard *Machine Learning Systems* : mlsysbook.ai.

— C. Guo et al. (Cornell), calibration ; Y. Geifman, R. El-Yaniv (Technion), classification sélective.